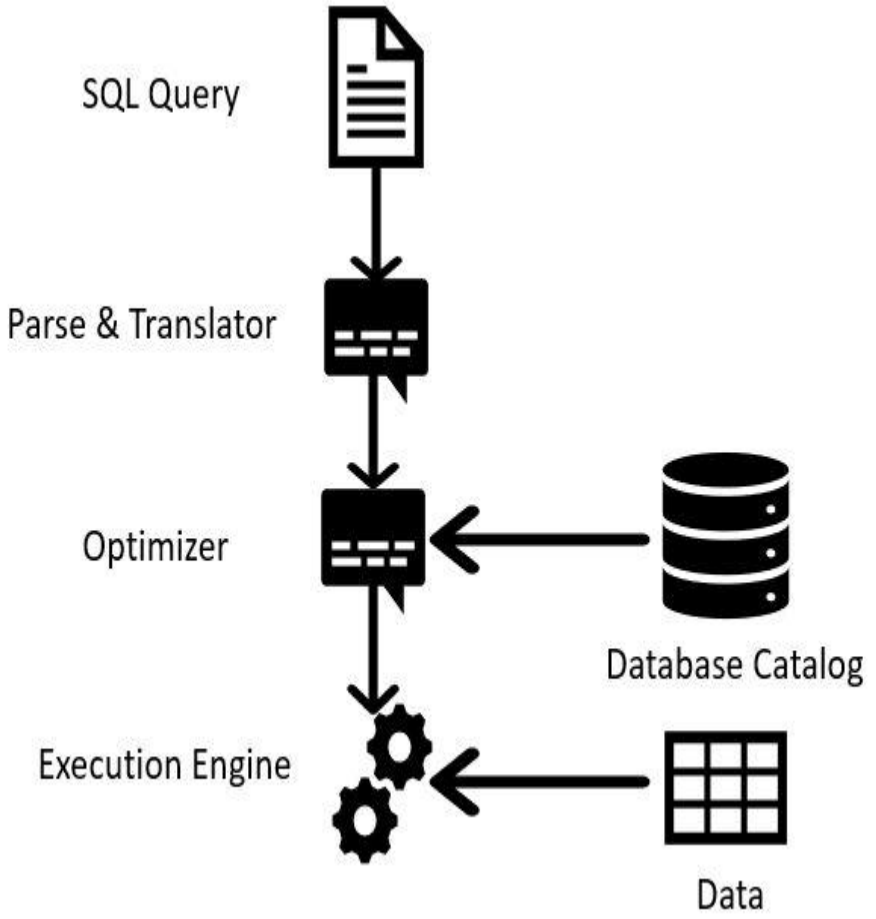
# Query Processing and Optimization

Dr Ilayaraja N

# Introduction

- DBMS techniques to process a query
  - Scanner identifies query tokens
  - Parser checks the query syntax
  - Validation checks all attribute and relation names
  - Query tree (or query graph) created
  - Execution strategy or query plan devised
- Query optimization
  - Planning a good execution strategy



# Introduction (cont’d.)

- A query expressed in a high-level query language such as SQL must first be scanned,

  parsed, and validated.
- The scanner identifies the query tokens—such as SQL keywords, attribute names, and relation names—that appear in the text of the query,
- the parser checks the query syntax to determine whether it is formulated according to the syntax rules (rules of grammar) of the query language.
- The query must also be validated by checking that all attribute and relation names are valid

  and semantically meaningful names in the schema of the particular database being queried.

# Introduction (cont’d.)

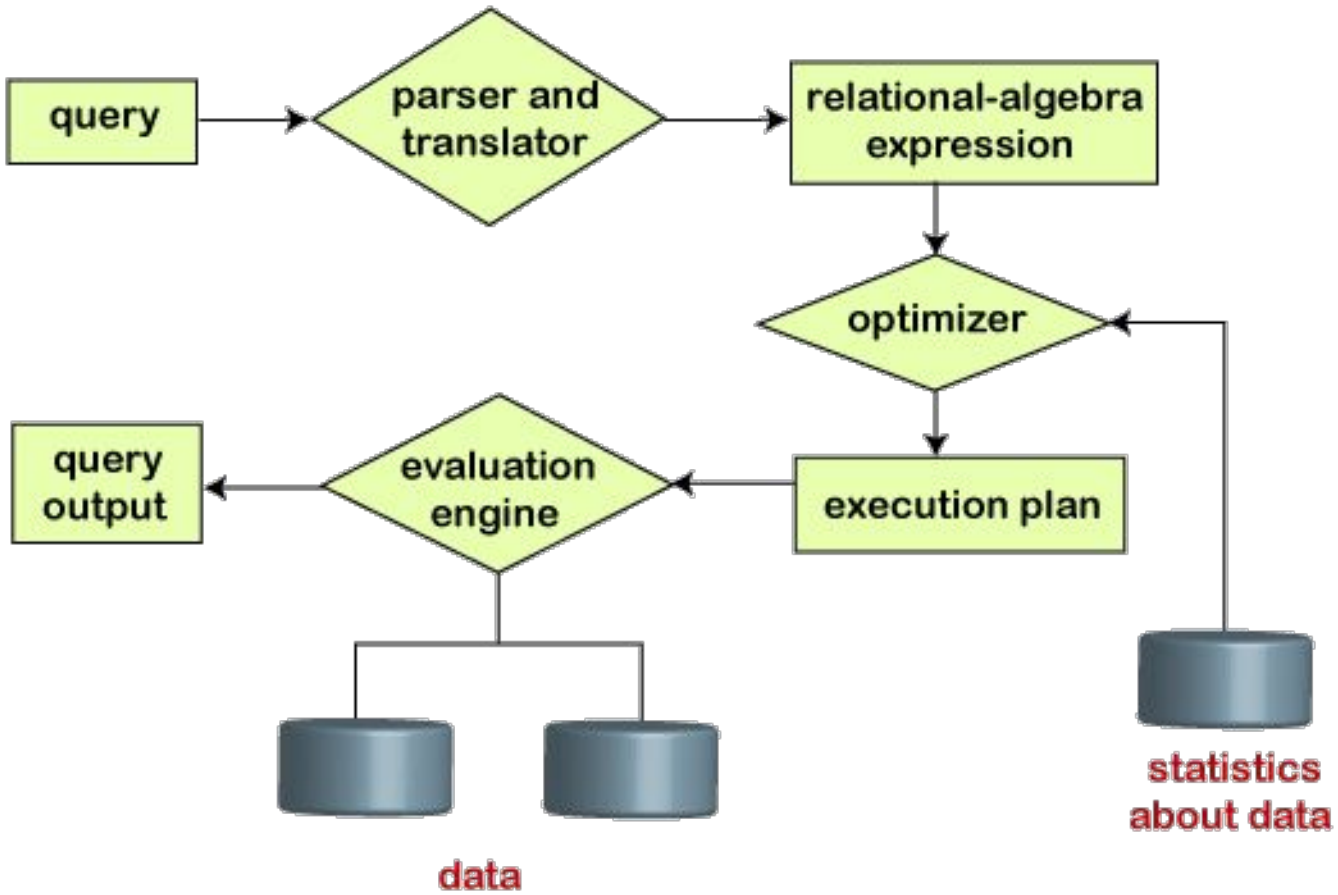
- An internal representation of the query is then created, usually as a tree data structure called a query tree.
- It is also possible to represent the query using a graph data structure called a query graph, which is generally a directed acyclic graph (DAG).
- The DBMS must then devise an execution strategy or query plan for retrieving the results of the query from the database files.
- A query has many possible execution strategies, and the process of choosing a suitable one for processing a query is known as query optimization.
- It focus on how queries are processed and what algorithms are used to perform individual operations within the query.

# Basic Steps in Query Processing (cont.)

- **Parser and translator**
  - Parser checks syntax (e.g., correct relation and operator names)
  - Translate the (SQL) query into relational algebra
  - Translate the query into its internal form.

- **Query Optimizer**
  - The process of choosing a suitable **execution strategy** for processing a query.
  - Chooses the most efficient implementation to execute the query
    - Produces equivalent relational algebra expressions
    - Annotates them with instructions (algorithms): query execution plan (QEP)
    - Estimates the cost of each equivalent QEP, according to a given cost model
    - Choose the “best” QEP
- **Evaluation engine**
  - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query

# Basic Steps in Query Processing




Steps in query processing

1. Parsing and translation
2. Optimization
3. Query Evaluation Engine

# Query Processing

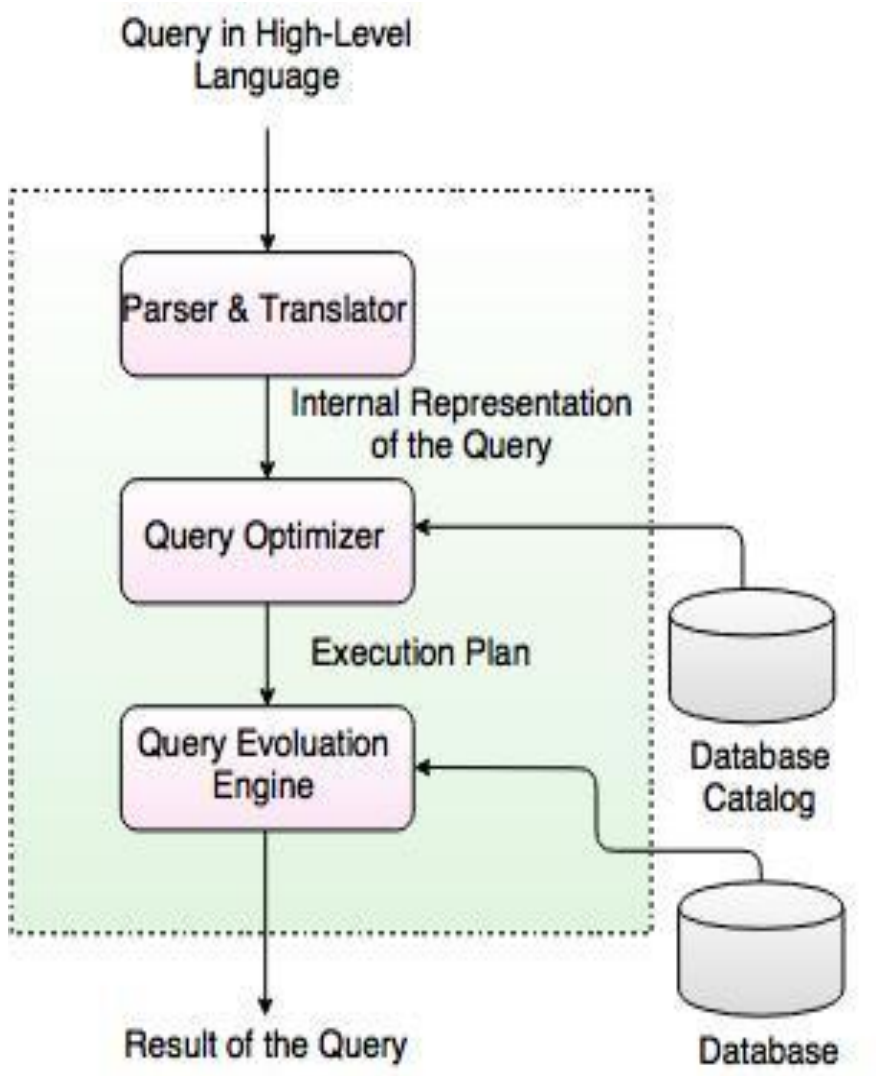
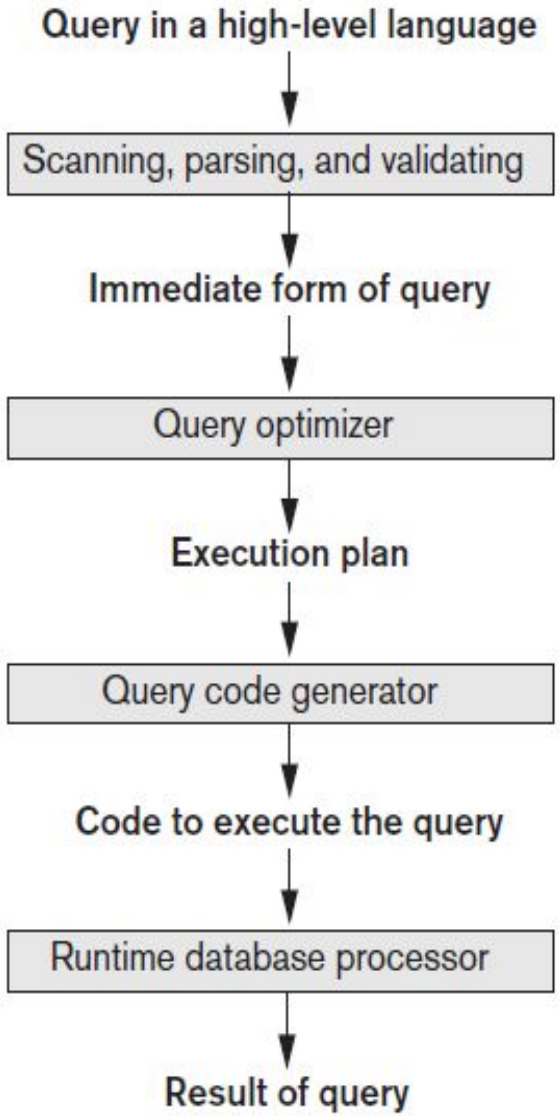



Fig. Query Processing




**Code can be:**

Executed directly (interpreted mode)

Stored and executed later whenever needed (compiled mode)

# Basic Steps in Query Processing (cont.)

- The query optimizer module has the task of producing a good execution plan, and the code generator generates the code to execute that plan.
- The runtime database processor has the task of running (executing) the query code, whether in compiled or interpreted mode, to produce the query result.
- If a runtime error results, an error message is generated by the runtime database processor.

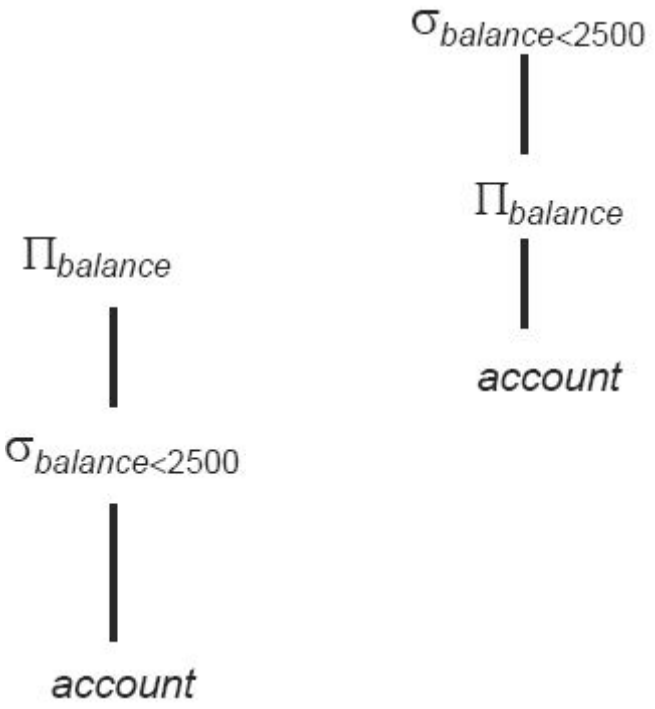
# Basic Steps in Query Processing

- Two internal representations of a query:
  - Query Tree
  - Query Graph

- `Select balance From account where balance < 2500`
  - $\sigma_{balance<2500}(\prod_{balance}(account))$

is equivalent to

- $\prod_{balance}(\sigma_{balance<2500}(account))$



# Basic Steps in Query Processing

- **Annotated relational algebra expression**
  - Specifying detailed evaluation strategy is called an **evaluation-plan**.
  - E.g., can use an index on **balance** to find accounts with **balance < 2500**,
  - or can perform complete relation scan and discard accounts with **balance ≥ 2500**

- **Query Optimization**:
  - Amongst all equivalent evaluation plans choose the one with lowest cost.
  - Cost is estimated using statistical information from the database catalog
    - e.g. number of tuples in each relation, size of tuples

# Basic Steps: Optimization

- 1st level of optimization: an SQL query has many equivalent relational algebra expressions
  - $\sigma_{salary<75000}(\prod_{salary}(instructor))$ *and* $\prod_{salary}(\sigma_{salary<75000}(instructor))$ are equivalent
  - They both correspond to SELECT salary
    FROM instructor
    WHERE salary < 75000
- 2nd level of optimization: a relational algebra operation can be evaluated using one of several different algorithms
  - e.g., block nested-loop join VS. merge-join; file scan VS. index scan
- Input of optimization: a query in the form of an algebra expression
- Output of optimization: the "best" annotated relational algebra expression specifying detailed evaluation strategy **(query evaluation plan or query execution plan – QEP)** answering the input query

# Basic Steps: Optimization (Cont.)

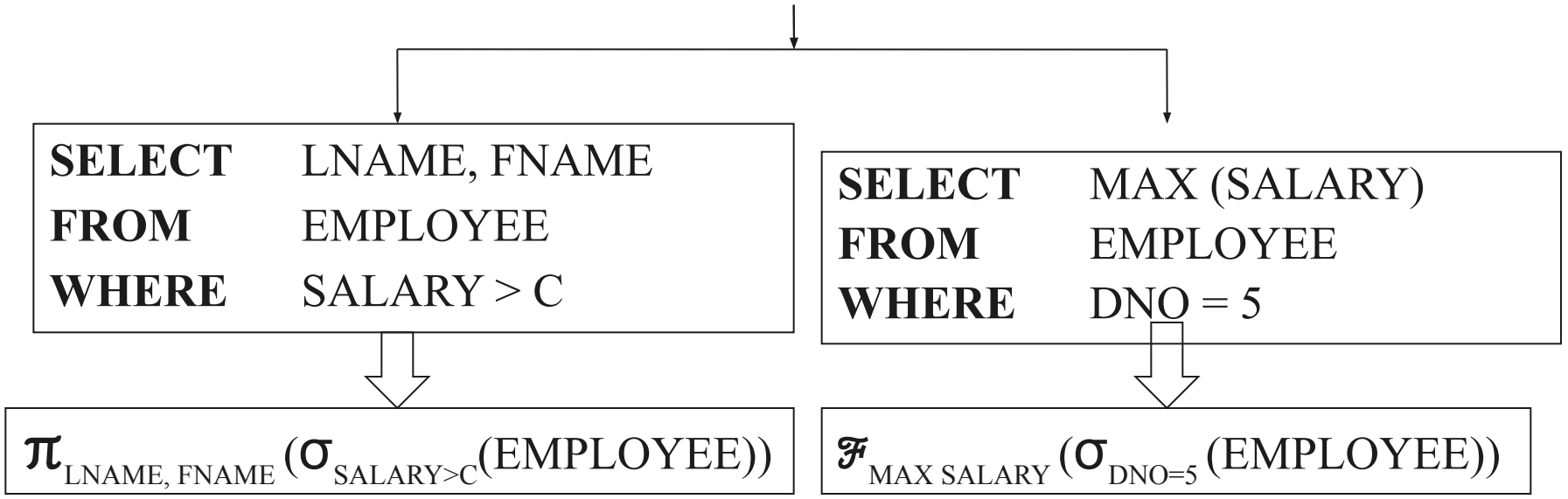
- Different query evaluation plans have different costs
  - User is not expected to specify least-cost plans
- **Query Optimization:** amongst all equivalent QEP choose the one with lowest cost
  - Cost is estimated using statistical information from the database catalog
    - # of tuples in relations, tuple sizes, # of distinct values for a given attribute, etc.
- Evaluation of QEP)
  - How to measure query costs (establish a cost model)
  - Algorithms for evaluating relational algebra operations and their cost
  - How to combine algorithms for individual operations in order to evaluate a complex expression (QEP)
- Choosing the best QEP)
  - How to optimize queries, that is, how to find a QEP with lowest estimated cost

# Translating SQL Queries into Relational Algebra and Other Operators

- SQL
  - Query language used in most RDBMSs
- Query decomposed into query blocks
  - Basic units that can be translated into the algebraic operators
  - Contains single SELECT-FROM-WHERE expression
    - May contain GROUP BY and HAVING clauses

# Translating SQL Queries into Relational Algebra (2)

```
SELECT    LNAME, FNAME
FROM      EMPLOYEE
WHERE     SALARY > (SELECT    MAX (SALARY)
                    FROM      EMPLOYEE
                    WHERE     DNO = 5);
```




Copyright © 2016 Ramez Elmasri and Shamkant B. Navathe

# Translating SQL Queries (cont'd.)

- Example:

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ( SELECT MAX (Salary)
                 FROM   EMPLOYEE
                 WHERE  Dno=5 );
```

  - Inner block

```
( SELECT MAX (Salary)
  FROM   EMPLOYEE
  WHERE  Dno=5 )
```

  - Outer block

```
SELECT  Lname, Fname
FROM    EMPLOYEE
WHERE Salary > c
```

# Translating SQL Queries (cont'd.)

- Example (cont'd.)
  - Inner block translated into:

$$\Im_{\text{MAX Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$$

  - Outer block translated into:

$$\pi_{\text{Lname,Fname}}(\sigma_{\text{Salary}>c}(\text{EMPLOYEE}))$$

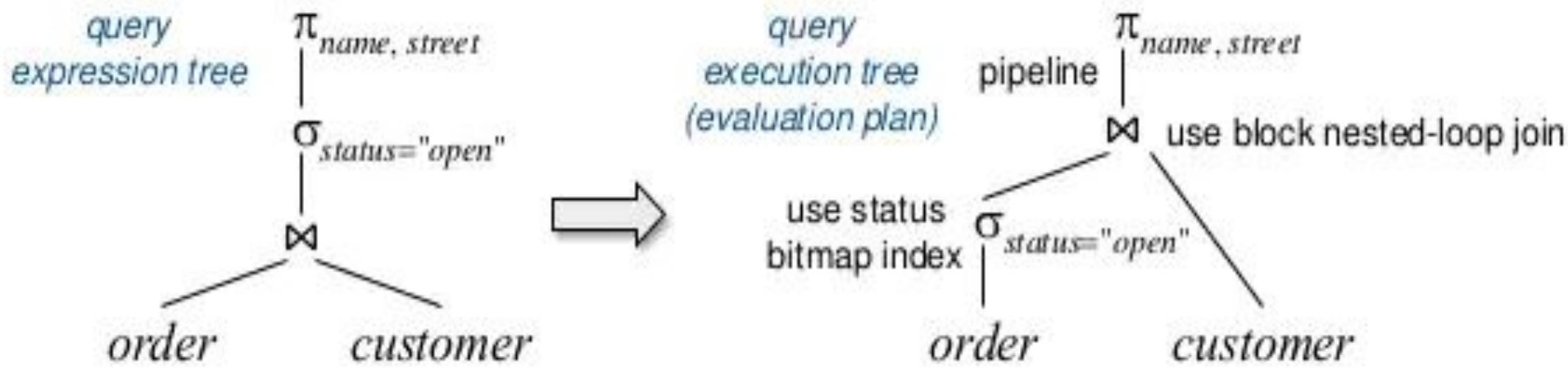
- Query optimizer chooses execution plan for each query block

# Query Expression and Execution

```
SELECT name, street
FROM Customer, Order
WHERE Order.customerID = Customer.customerID AND status = 'open';
```

- Transform the SQL query to the following query plan

$$\pi_{name,\ street}(\sigma_{status="open"}(order \bowtie customer))$$



# Measures of Query Cost

Response time (wall-clock time needed to execute a plan) depends on several factors

- system configuration
  - amount of dedicated buffer in RAM (aka, memory, main memory)
  - whether or not indices are (partially) stored permanently in the buffer
- runtime conditions
  - amount of free buffer at the time the plan is executed
  - content of the buffer at the time the plan is executed
  - parameters, embedded in queries, which are resolved at runtime only

    *SELECT salary*
    *FROM instructor*
    *WHERE salary < $a*

    where $a is a variable provided by the application (user)

Thus

1. cost models (like ours) focus on resource consumption rather than response time (optimizers minimize resource consumption rather than response time)
2. different optimizers may make different assumptions (parameters): every theoretical analysis must be recast with the actual parameters used by the concrete system (optimizer) to which the analysis is going to be applied

# Measures of Query Cost (Cont.)

- Query cost (total elapsed time for answering a query) is measured in terms of different resources
  - *disk access* (I/O operation on disk)
  - *CPU usage*
  - (*network communication* for distributed DBMS – later in this course)
- Typically **disk access** is the predominant cost, and is also relatively easy to estimate. Measured by taking into account
  - **Number of seeks** (number of random I/O accesses)
  - **Number of blocks read**
  - **Number of blocks written**
    - It is generally assumed cost for writing to be twice as the cost for reading (data is read back after being written to ensure the write was successful)

**VERY IMPORTANT!!!**
- "**disk**" refers to **permanent drive** for file storage, **hard-disk**, **secondary memory**, **permanent memory**
- "**memory**" refers to **volatile drive** for data storage, **RAM**, **main memory**, **buffer**

**These are all used as synonims**

This is a **so far** accepted choice for measuring query costs (**cost model**).
New technologies: faster hard-disks (solid-state drives – SSD) and cheaper (thus bigger) RAM might direct towards different cost models (e.g., based also on CPU usage or RAM I/O operations)